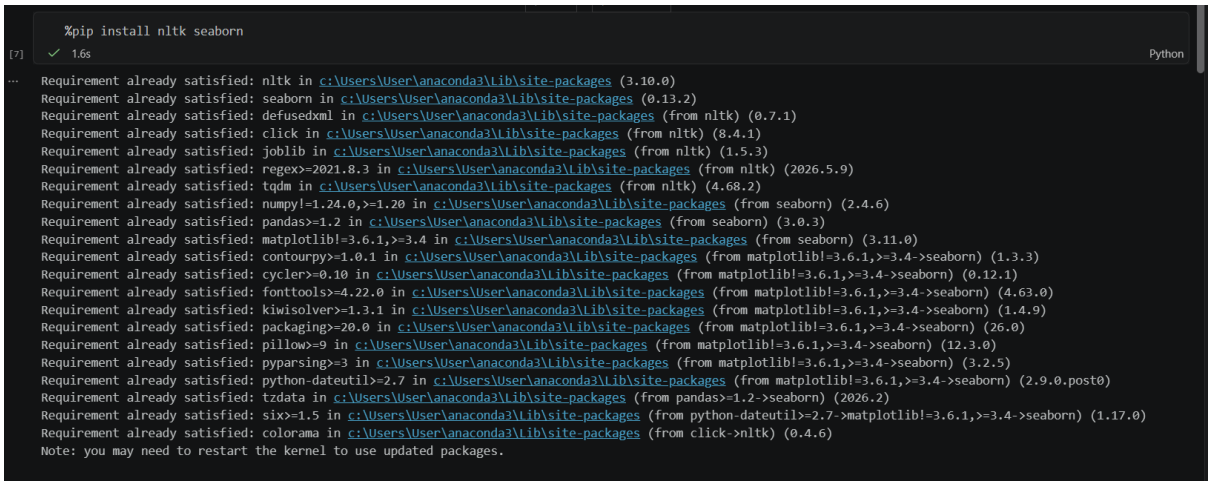
LAPORAN PRAKTIKUM
KECERDASAN BUATAN

Text Classification

Nama : Joyce Paulina Simanjuntak
NIM : 42324059
Prodi : DIII-Teknologi Informasi

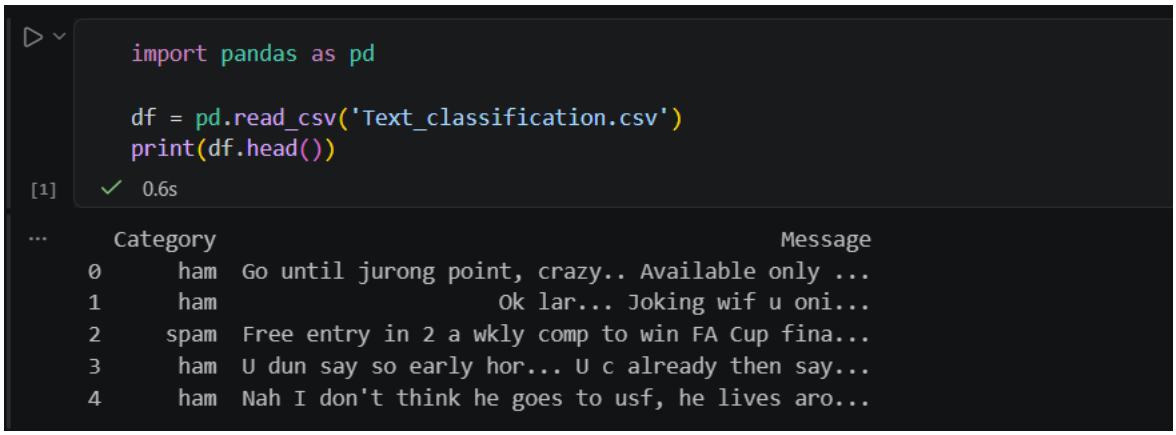
INSTITUT TEKNOLOGI DEL
FAKULTAS VOKASI
TAHUN AJARAN
2026/2027

Text Classification



```
%pip install nltk seaborn
[7] ✓ 1.6s                                                                    Python
...
Requirement already satisfied: nltk in c:\Users\User\anaconda3\Lib\site-packages (3.10.0)
Requirement already satisfied: seaborn in c:\Users\User\anaconda3\Lib\site-packages (0.13.2)
Requirement already satisfied: defusedxml in c:\Users\User\anaconda3\Lib\site-packages (from nltk) (0.7.1)
Requirement already satisfied: click in c:\Users\User\anaconda3\Lib\site-packages (from nltk) (8.4.1)
Requirement already satisfied: joblib in c:\Users\User\anaconda3\Lib\site-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in c:\Users\User\anaconda3\Lib\site-packages (from nltk) (2026.5.9)
Requirement already satisfied: tqdm in c:\Users\User\anaconda3\Lib\site-packages (from nltk) (4.68.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in c:\Users\User\anaconda3\Lib\site-packages (from seaborn) (2.4.6)
Requirement already satisfied: pandas>=1.2 in c:\Users\User\anaconda3\Lib\site-packages (from seaborn) (3.0.3)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in c:\Users\User\anaconda3\Lib\site-packages (from seaborn) (3.11.0)
Requirement already satisfied: contourpy>=1.0.1 in c:\Users\User\anaconda3\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\Users\User\anaconda3\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\Users\User\anaconda3\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\Users\User\anaconda3\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.9)
Requirement already satisfied: packaging>=20.0 in c:\Users\User\anaconda3\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (26.0)
Requirement already satisfied: pillow>=9 in c:\Users\User\anaconda3\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (12.3.0)
Requirement already satisfied: pyparsing>=3 in c:\Users\User\anaconda3\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in c:\Users\User\anaconda3\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
Requirement already satisfied: tzdata in c:\Users\User\anaconda3\Lib\site-packages (from pandas>=1.2->seaborn) (2026.2)
Requirement already satisfied: six>=1.5 in c:\Users\User\anaconda3\Lib\site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
Requirement already satisfied: colorama in c:\Users\User\anaconda3\Lib\site-packages (from click->nltk) (0.4.6)
Note: you may need to restart the kernel to use updated packages.
```

Penjelasan: Baris ini tugasnya nginstall dua library tambahan langsung dari dalam *notebook* yaitu nltk dan *seaborn*. Nltk itu buat ngolah teks, misalnya bersihin kata, misahin kalimat jadi per kata, atau ngilangin kata-kata umum yang nggak penting. *Seaborn* buat bikin grafik yang tampilannya lebih rapi dan kekinian dibanding matplotlib biasa, jadi cocok buat visualisasi data. Tanda %pip itu perintah khusus *Jupyter Notebook*, jadi *install*nya nyambung ke *environment* yang lagi dipakai, bukan ke tempat lain. Cukup dijalanin sekali aja, dan kalau *library*-nya udah pernah keinstall, nanti cuma muncul keterangan kalau semuanya udah terpenuhi.



```
import pandas as pd

df = pd.read_csv('Text_classification.csv')
print(df.head())
[1] ✓ 0.6s
...
  Category                                            Message
0      ham  Go until jurong point, crazy.. Available only ...
1      ham                      Ok lar... Joking wif u oni...
2     spam  Free entry in 2 a wkly comp to win FA Cup fina...
3      ham  U dun say so early hor... U c already then say...
4      ham  Nah I don't think he goes to usf, he lives aro...
```

Penjelasan: Kodenya buka file Text_classification.csv pakai pandas, terus isinya disimpan ke df dalam bentuk tabel. Habis itu print(df.head()) nampilin 5 baris paling atas, jadi kita bisa langsung lihat ada kolom apa aja (biasanya kolom teks dan kolom label kategorinya) dan isinya kayak gimana sebelum diolah lebih jauh. Karena ini cuma 5 baris pertama, jumlah total datanya belum kelihatan, itu nanti dicek pakai kode lain. Satu hal yang perlu diingat yaitu df yang tadinya isi data *Mall Customers* sekarang ketimpa data teks ini. Kalau suatu saat butuh data *mall* lagi, harus dibaca ulang dari file aslinya. Selain itu, karena pakai print, hasilnya muncul sebagai teks polos, bukan tabel rapi kayak kalau cuma nulis df.head() di *notebook*.

```python
# Download resource NLTK yang diperlukan
import nltk
nltk.download('punkt_tab')
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')

# Import library untuk preprocessing
import re
import string
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

# Inisialisasi lemmatizer
lemmatizer = WordNetLemmatizer()

# Fungsi preprocessing teks
def clean_text(text):
    text = text.lower()  # Ubah menjadi lowercase
    text = re.sub(r'\d+', '', text)  # Hapus angka
    text = text.translate(str.maketrans('', '', string.punctuation))  # Hapus tanda baca
    words = word_tokenize(text)  # Tokenisasi
    words = [word for word in words if word not in stopwords.words('english')]  # Hapus stopwords
    words = [lemmatizer.lemmatize(word) for word in words]  # Lemmatization
    return ' '.join(words)

# Terapkan preprocessing ke teks
df['clean_text'] = df['Message'].apply(clean_text)

# Cek hasil preprocessing
print("\nHasil Preprocessing:\n", df[['Message', 'clean_text']].head())
```

[2] ✓ 59.8s

Output:

```
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\User\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping tokenizers\punkt_tab.zip.
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\User\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping tokenizers\punkt.zip.
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\User\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping corpora\stopwords.zip.
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\User\AppData\Roaming\nltk_data...

Hasil Preprocessing:
                                              Message  \
0  Go until jurong point, crazy.. Available only ...
1                      Ok lar... Joking wif u oni...
2  Free entry in 2 a wkly comp to win FA Cup fina...
3  U dun say so early hor... U c already then say...
4  Nah I don't think he goes to usf, he lives aro...

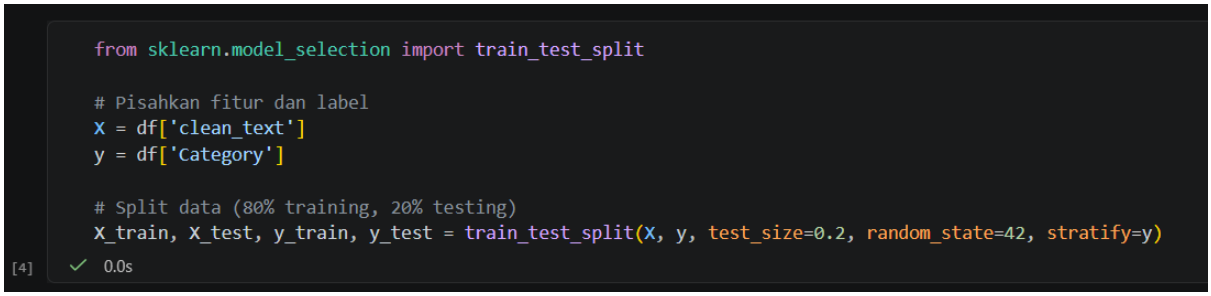
                                          clean_text
0  go jurong point crazy available bugis n great ...
1                            ok lar joking wif u oni
2  free entry wkly comp win fa cup final tkts st ...
3                u dun say early hor u c already say
4           nah dont think go usf life around though
```

Penjelasan: Bagian ini buat bersihin teks di kolom *Message* biar siap dipakai buat klasifikasi. Pertama, *resource* yang dibutuhkan nltk di*download* dulu, *tokenizer* (buat misahin kalimat jadi kata per kata), daftar *stopwords*, dan *wordnet* buat *lemmatization*. Habis itu dibuat fungsi

clean_text yang isinya urutan bersih-bersih. Semua huruf diubah jadi kecil, angka dibuang, tanda baca dihapus, lalu teksnya dipecah jadi kata-kata. Kata umum bahasa Inggris yang nggak bawa makna penting, kayak “the”, “is”, “and”, dibuang juga (itu yang disebut *stopwords*). Sisanya diubah ke bentuk dasarnya lewat *lemmatization*, misalnya “running” jadi “run” atau “messages” jadi “message”, biar kata yang sebenarnya sama nggak dianggap beda. Terakhir, kata-kata itu digabung lagi jadi satu kalimat. Fungsi ini lalu dijalankan ke setiap baris di kolom *Message* pakai *apply*, dan hasilnya disimpan di kolom baru bernama clean_text. Di akhir, *print* nampilin 5 baris pertama dengan kolom *Message* dan clean_text berdampingan, jadi kita bisa bandingin teks asli sama versi yang udah bersih. Ada satu catatan yaitu kodenya mengasumsikan ada kolom bernama *Message* di file CSV-nya. Jadi kalau namanya beda, bakal *error*.

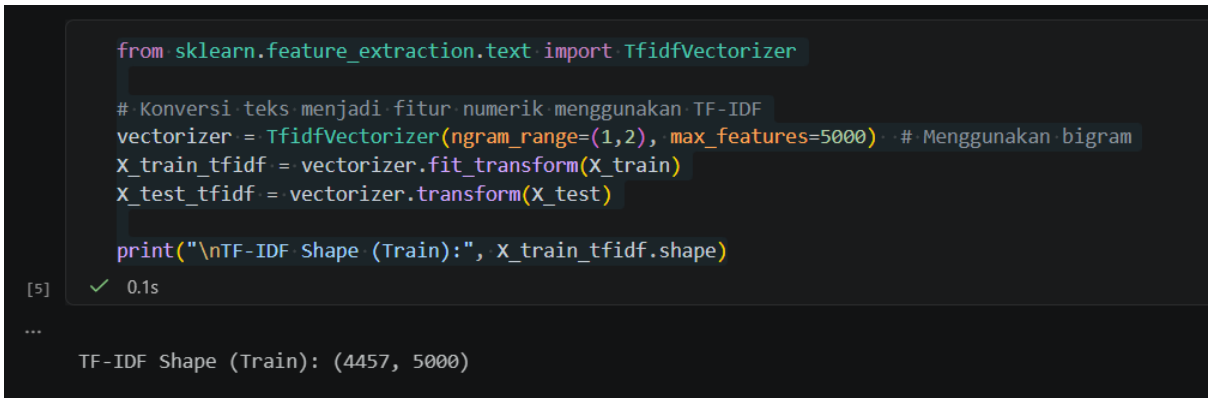


Penjelasan: Model *machine learning* nggak ngerti tulisan “ham” atau “*spam*”, jadi label di kolom *Category* diubah dulu jadi angka. Misal: ham (pesan biasa) jadi 0, *spam* jadi 1. Hasilnya langsung nimpa kolom yang sama, jadi tulisan aslinya hilang. Setelah itu value_counts() ngitung ada berapa pesan di masing-masing label, jadi kelihatan datanya seimbang atau nggak. Biasanya ham jauh lebih banyak dari *spam*, dan ini penting diperhatiin karena model bisa jadi lebih sering nebak ham. Tapi harus hati-hati juga, kalau ada isi kolom yang tulisannya beda sedikit dari ‘ham’ atau ‘*spam*’ (misalnya huruf besar atau ada spasi), hasilnya jadi NaN. Dan kalau sel ini dijalanin dua kali, semua isinya jadi NaN karena angka 0 dan 1 nggak cocok lagi sama ‘ham’/’*spam*’.



Penjelasan: Di sini data dipisah dulu jadi dua bagian sebelum dipakai buat bikin model. X diisi teks yang udah dibersihin (kolom clean_text) sebagai bahan yang dipelajari model, sedangkan y diisi labelnya (0 buat ham, 1 buat *spam*) sebagai jawaban yang bener. Terus train_test_split ngebagi semuanya jadi 80% buat latihan (X_train, y_train) dan 20% buat ngetes (X_test, y_test). Bagian tes ini sengaja disimpan dan nggak dikasih lihat ke model pas belajar, jadi nanti bisa dipakai buat ngecek model beneran paham atau cuma hafal. random_state=42

bikin pembagiannya selalu sama tiap dijalanin, dan stratify=y mastiin perbandingan ham sama *spam* di data latihan dan data tes tetap mirip kayak data aslinya, ini penting karena *spam* jumlahnya jauh lebih sedikit.



Penjelasan: Teks itu nggak bisa langsung dimakan model, jadi di sini diubah jadi angka pakai TF-IDF. Intinya, tiap kata dikasih bobot jadi kata yang sering muncul di sebuah pesan tapi jarang muncul di pesan lain dapat nilai tinggi (berarti ciri khas pesan itu), sedangkan kata yang ada di mana-mana nilainya kecil. *Setting*nya ada dua. ngram_range=(1,2) artinya yang dihitung bukan cuma kata tunggal, tapi juga pasangan dua kata berurutan, misalnya "free entry" atau "claim prize", karena frasa kayak gitu sering jadi tanda *spam*. Lalu max_features=5000 membatasi cuma 5000 kata/frasa paling penting yang dipakai, biar modelnya nggak keberatan. Soal alurnya, fit_transform dipakai ke data latihan yaitu *vectorizer* belajar kosakata dan bobotnya dari situ, sekalian ngubah teksnya jadi angka. Data tes cuma di-*transform*, jadi pakai kosakata yang sama dari data latihan tanpa belajar ulang. Ini sengaja supaya data tes benar-benar "belum pernah dilihat" model. Terakhir, print nampilin ukuran hasilnya (X_train_tfidf.shape), yaitu jumlah baris data latihan dan jumlah kolom fitur, yang maksimal 5000.

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Akurasi
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)

# Classification Report
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

[7] ✓ 0.0s

```
Accuracy: 0.979372197309417

Classification Report:
              precision    recall  f1-score   support

           0       0.98      1.00      0.99       966
           1       0.98      0.86      0.92       149

    accuracy                           0.98      1115
   macro avg       0.98      0.93      0.95      1115
weighted avg       0.98      0.98      0.98      1115
```

Penjelasan: Bagian ini buat ngecek seberapa bagus model nebak data tes. Yang diimpor tiga alat evaluasi dari *sklearn*, tapi di kode ini yang kepake cuma dua: accuracy_score dan classification_report. confusion_matrix belum dipanggil, mungkin buat sel berikutnya. accuracy_score ngebandingin jawaban asli (y_test) sama tebakan model (y_pred), terus ngitung persentase yang bener. Misalnya hasilnya 0.97, berarti 97% pesan di data tes ditebak dengan benar. Tapi akurasi aja bisa nipu, karena *spam* jumlahnya jauh lebih sedikit dari ham. Model yang asal nebak "ham" terus pun bisa kelihatan akurat. Makanya ada classification_report yang ngasih rincian per kelas (0 dan 1): precision (dari semua yang ditebak *spam*, berapa yang beneran *spam*), *recall* (dari semua *spam* asli, berapa yang berhasil ketangkep), f1-score (gabungan keduanya), dan support (jumlah data asli di tiap kelas). Buat kasus *spam*, *recall* dan *precision* di baris kelas 1 yang paling perlu diperhatiin.

Satu catatan: y_pred belum dibuat di kode yang kamu kirim sampai sekarang. Kalau sel ini dijalanin langsung, bakal *error* NameError. Biasanya y_pred dihasilkan dari model yang udah dilatih, lewat sesuatu kayak model.predict(X_test_tfidf), jadi pastiin langkah *training* dan prediksinya udah dijalanin dulu.

```python
# Confusion Matrix
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.metrics import confusion_matrix

# Hitung confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)

# Definisi elemen dalam confusion matrix
TN, FP, FN, TP = conf_matrix.ravel()

# Buat DataFrame confusion matrix
df_conf_matrix = pd.DataFrame(
    conf_matrix,
    index=['Actual Ham', 'Actual Spam'],
    columns=['Predicted Ham', 'Predicted Spam']
)

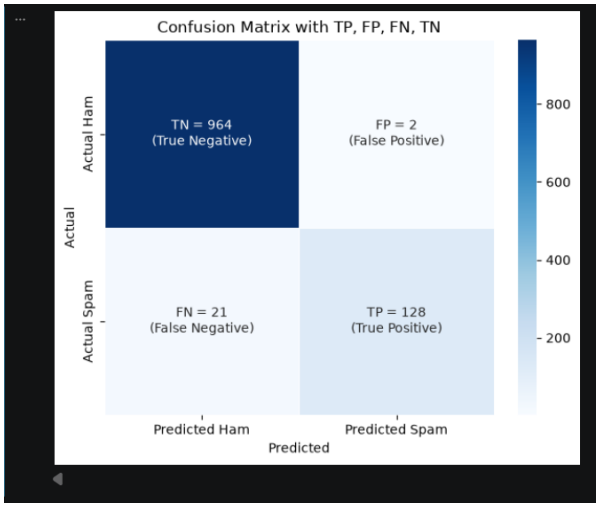
# Label untuk TN, FP, FN, TP
labels = np.array([
    [f"TN = {TN}\n(True Negative)", f"FP = {FP}\n(False Positive)"],
    [f"FN = {FN}\n(False Negative)", f"TP = {TP}\n(True Positive)"]
])

# Plot heatmap dengan keterangan
plt.figure(figsize=(6, 5))
sns.heatmap(
    df_conf_matrix,
    annot=labels,
    fmt='',
    cmap='Blues',
    linewidths=0.5,
    cbar=True
)

# Tambahkan label dan judul
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix with TP, FP, FN, TN')
plt.tight_layout()
plt.show()
```

[9] ✓ 1.0s

Output:



Penjelasan: Kalau tadi evaluasinya berupa angka, sekarang hasilnya dibikin jadi gambar biar gampang dibaca. confusion_matrix ngitung tabel 2x2 yang isinya hasil tebakan model dibandingin sama jawaban asli, lalu ravel() memecahnya jadi empat angka terpisah yaitu TN, FP, FN, dan TP. Arti keempatnya begini. TN itu ham yang ditebak ham (bener). TP itu *spam* yang ditebak *spam* (benar). FP itu ham yang salah ditebak *spam*, jadi pesan biasa malah masuk folder *spam*. FN itu *spam* yang lolos karena ditebak ham. Dua yang terakhir adalah kesalahan model. Angka-angkanya lalu dimasukkan ke tabel pandas dengan nama baris dan kolom yang

jelas (Actual Ham, Actual *Spam*, Predicted Ham, Predicted *Spam*), dan dibuat teks keterangan buat tiap kotak, misalnya “TP = 140 (True Positive)”. Setelah itu sns.heatmap menggambar semuanya jadi kotak-kotak berwarna biru, makin tua warnanya berarti makin banyak datanya, dengan teks keterangan tadi ditaruh di dalam kotaknya. Sisanya cuma ngasih label sumbu, judul, dan nampilin gambarnya. Biasanya kotak TN paling gelap karena ham jumlahnya banyak. Buat ngecek kualitas model, yang paling perlu dilihat itu kotak FN dan FP, semakin kecil semakin bagus. Sama kayak sebelumnya, y_pred harus udah ada dulu sebelum sel ini dijalanin.

```
# Contoh teks baru
new_texts = ["Congratulations! You won a free trip to Bali.",
             "Hey, are you coming to the party tonight?"]

# Preprocess teks baru
new_texts_clean = [clean_text(text) for text in new_texts]

# Transformasi dengan TF-IDF
new_texts_tfidf = vectorizer.transform(new_texts_clean)

# Prediksi
predictions = model.predict(new_texts_tfidf)

# Output hasil prediksi
for text, label in zip(new_texts, predictions):
    print(f"\nMessage: {text} \nPredicted: {'Spam' if label == 1 else 'Ham'}")
```

[10] ✓ 0.0s

```
Message: Congratulations! You won a free trip to Bali.
Predicted: Spam

Message: Hey, are you coming to the party tonight?
Predicted: Ham
```

Penjelasan: Di sini model dites pakai dua pesan baru buat lihat apakah dia bisa bedain *spam* dan pesan biasa. Satu pesan isinya ucapan selamat dapat hadiah liburan gratis ke Bali, satunya lagi ajakan ke pesta malam ini. Dua-duanya harus diperlakukan persis kayak data latihan dulu: dibersihin pakai clean_text, terus diubah jadi angka lewat vectorizer.transform. Pakai *transform* aja, bukan fit_transform, karena kosakatanya harus sama dengan yang dipelajari waktu latihan. Setelah itu model.predict ngeluarin tebakan 0 atau 1 buat tiap pesan, dan perulangan di bawahnya nyetak pesan aslinya beserta hasilnya, di mana 1 jadi “*Spam*” dan 0 jadi “Ham”. Harapannya pesan hadiah gratis kedeteksi *Spam* dan ajakan pesta kedeteksi Ham. Lalu hal yg harus diingat, model harus udah dibuat dan dilatih di sel sebelumnya, kalau belum bakal muncul NameError.